Panchip Microelectronics Co., Ltd.

PAN3029/3060 Series Early Interrupt Application Reference

Current version: 1.1

Release date: 2024.04

Shanghai Panqi Microelectronics Co., Ltd.

Address: Room 302, Building D, No. 666, Shengxia Road, Zhangjiang Hi-Tech Park, Shanghai

Phone number: 021-50802371

Website：http://www.panchip.com

Document Description

Due to version upgrades or other reasons, the content of this document will be updated from time to time. Unless otherwise agreed, the content of this document is only for guidance, and all statements, information and suggestions in this document do not constitute any express or implied warranty.

Trademarks

Panqi is a trademark of Panqi Microelectronics Co., Ltd. Other names mentioned in this document are trademarks/registered trademarks of their respective owners.

Disclaimer

All or part of the products, services or features described in this document may not be within the scope of your purchase or use. Unless otherwise agreed in the contract, Panqi Microelectronics Co., Ltd. does not make any express or implied statements or warranties regarding the contents of this document.

Revision History

Version	Revision Date	Description
V1.0	2023.06	Initial version
V1.1	2024.04	Modify function interface and chip description

Table of contents

1 Function Introduction

The PAN3029/3060 series chips provide an early interrupt function. The early interrupt function is to check the decoded data during the chip reading a frame of data, determine whether it is what you want, and then decide whether to continue reading or abandon this frame of data.

The flow chart is as follows:

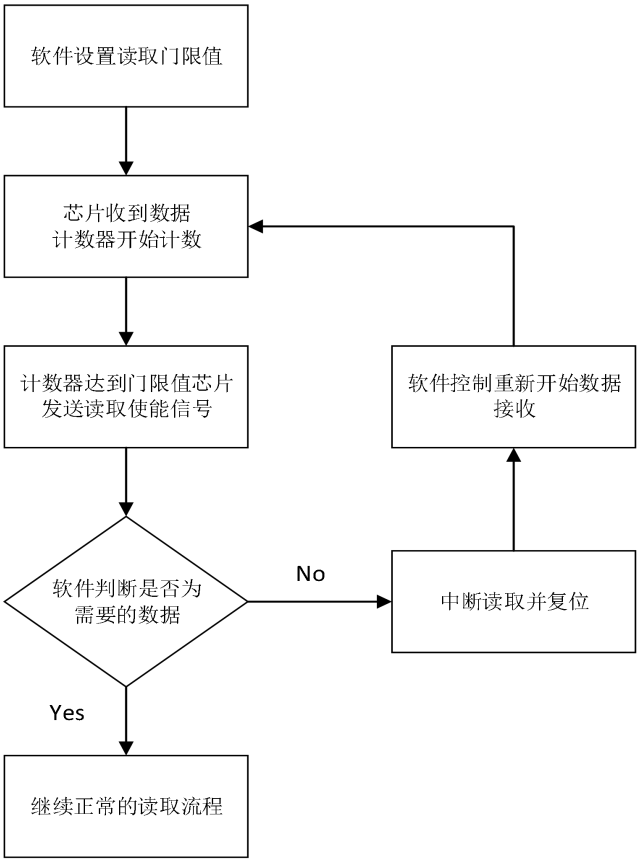

Figure 1-1 Early interruption flow chart

2 Software Design Reference

2.1 Software Design Process

1. Initialize the chip.

2. Configure RF parameters.

3. Configure the chip to early interrupt mode, and set the early interrupt start byte and header length (header length also called counter threshold) through registers. The start byte refers to the byte from which the early interrupt starts to check, and the header length refers to the number of bytes of data checked by the early interrupt (the header length only supports 8 bytes or 16 bytes, represented by PLHD_LEN8/PLHD_LEN16 respectively).

4. Enter the receiving mode.

5. The chip receives data, and the internal counter starts counting. When a byte is received, it adds 1 until the counter reaches the header length. The chip generates an early interrupt signal for the software to read.

6. The software determines whether it is the data it wants. If so, it continues to read. If not, it stops reading.

2.2 Software Design Verification

2.2.1 Verification steps

1. The sending module periodically sends 100-byte data packets, the first 30 bytes of data are

```
uint8_t tx_test_buf[100] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```

2. Configure the receiving module to the early interrupt mode, and set it to check 16 bytes of data starting from the 5th byte;

```
rf_set_plhd_rx_on(5,PLHD_LEN8);
```

3. When the early interrupt signal is generated, print out the data obtained by the early interrupt;

4. Continue to receive and print out all the data of this frame;

5. Check the printing results through the serial port assistant.

2.2.2 SDK Examples

Main.c reference code:

```
ret = rf_init();
if(ret != OK)
{
    printf("    RF Init Fail");
    while(1);
}
rf_set_default_para();
rf_set_plhd_rx_on(5,PLHD_LEN16);
rf_enter_continous_rx();
while (1)
{
     rf_irq_process();
    if(rf_get_recv_flag() == RADIO_FLAG_PLHDRXDONE)
    {
        rf_set_recv_flag(RADIO_FLAG_IDLE);
        printf("###Plhd Rev :##\r\n");
        for(i = 0; i < RxDoneParams.PlhdSize; i++)
        {
            printf("0x%02x ", RxDoneParams.PlhdPayload[i]);
        }
        printf("\r\n");
    }

    if(rf_get_recv_flag() == RADIO_FLAG_RXDONE)
```

```
            {
                rf_set_recv_flag(RADIO_FLAG_IDLE);
                printf("Rx : SNR: %f ,RSSI: %f \r\n", RxDoneParams.Snr, RxDoneParams.Rssi);
                for(i = 0; i < RxDoneParams.Size; i++)
                {
                    printf("0x%02x ", RxDoneParams.Payload[i]);
                }
                printf("\r\n");
                cnt ++;
                printf("###Rx cnt %d##\r\n", cnt);
            }
            if((rf_get_recv_flag() == RADIO_FLAG_RXTIMEOUT) || (rf_get_recv_flag() == RADIO_FLAG_RXERR))
            {
                rf_set_recv_flag(RADIO_FLAG_IDLE);
                printf("Rxerr\r\n");
            }
        }
```

Radio.c reference code

```
    void rf_irq_process(void)
            if(irq & REG_IRQ_RX_PLHD_DONE)
            {
                RxDoneParams.PlhdSize = rf_get_plhd_len();
                RxDoneParams.PlhdSize = rf_plhd_receive(RxDoneParams.PlhdPayload, RxDoneParams.PlhdSize);
                irq &= ~REG_IRQ_RX_PLHD_DONE;
                rf_clr_irq(REG_IRQ_RX_PLHD_DONE);
                rf_set_recv_flag(RADIO_FLAG_PLHDRXDONE);
            }
```

The sample code configures the early interrupt mode and sets it to check 16 bytes of data starting from the 5th byte. After receiving the early interrupt signal, the main function chooses to print out the content received by the early interrupt and continue receiving; then the module will generate another receiving interrupt signal, and the main function will print out the complete received data content.

If you need to terminate the reception early, just execute rf_set_refresh(); after receiving the early interrupt signal. That is:

```
if(rf_get_recv_flag() == RADIO_FLAG_PLHDRXDONE)
{
    rf_set_recv_flag(RADIO_FLAG_IDLE);
    printf("###Plhd Rev :##\r\n");
    for(i = 0; i < RxDoneParams.PlhdSize; i++)
    {
        printf("0x%02x ", RxDoneParams.PlhdPayload[i]);
    }
    printf("\r\n");
    rf_set_refresh();//stop it
}
```

2.2.3 Verification Results

The serial port assistant displays the result as follows:

```
###Plhd Rev :##
0x05 0x06 0x07 0x08 0x09 0x0a 0x0b 0x0c 0x0d 0x0e 0x0f 0x10 0x11 0x12 0x13 0x00
Rx : SNR: 14.084321 ,RSSI: -60.000000
0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0a 0x0b 0x0c 0x0d 0x0e 0x0f 0x10
0x11 0x12 0x13 0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09
###Rx cnt 1##
###Plhd Rev :##
0x05 0x06 0x07 0x08 0x09 0x0a 0x0b 0x0c 0x0d 0x0e 0x0f 0x10 0x11 0x12 0x13 0x00
Rx : SNR: 16.130527 ,RSSI: -60.000000
0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0a 0x0b 0x0c 0x0d 0x0e 0x0f 0x10
0x11 0x12 0x13 0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09
###Rx cnt 2##
```

Figure 2-1 Serial port assistant display results

According to the results, the receiving module has an early interrupt, obtains the specified data, and continues to receive and receives the complete data packet.

2.3 Logic Analyzer Capture

2.3.1 Verification steps

1. The sending module periodically sends data packets.
2. The receiving module uses the early interrupt receiving mode and the normal receiving mode for receiving, and the two modes are received alternately.
3. Use the logic analyzer Channel 8 to capture the start and end signals of the sending end, and Channel 1 to capture the receiving end signal.

2.3.2 Verification Results

The capture results are shown in the figure below:

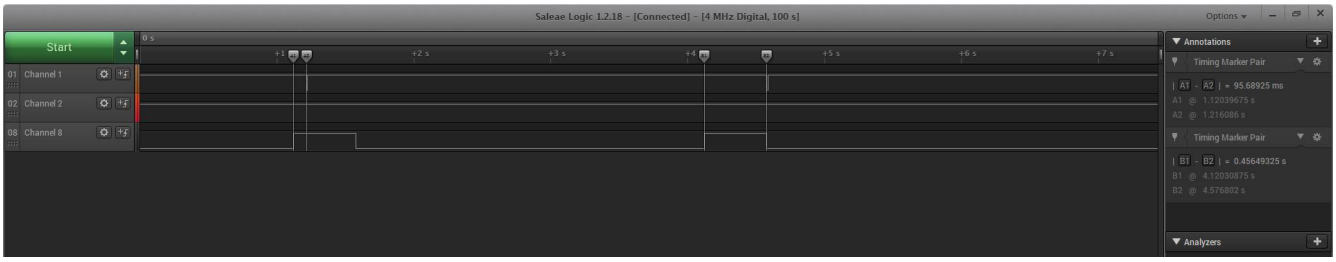

Figure 2-2 Capture results

From the results, it can be seen that the early interrupt reception mode generates an early interrupt at 95ms for the user to judge. The normal reception mode needs to generate a complete reception interrupt at 456ms.

2.4 Notes

The early interrupt function only supports reading two data lengths, 8 bytes/16 bytes, represented by PLHD_LEN8/PLHD_LEN16 respectively. Custom parameters cannot be used.

When the early interrupt function obtains data, the rf_plhd_receive() interface function is used, which is different from the rf_receive() interface function of the normal data packet. Its internal FIFO address is different.

2.5 Critical Case Testing

For critical cases, refer to the following test results (SF7, BW500K):

Transmitter configuration	Receiver configuration	Receive result

Transmitter packet length	Start byte of the early interrupt	Length of the early interruption header	Early interrupt result	Result of the receiving interrupt
100 bytes	80/81/82	PLHD_LEN16	Both	Both
100 bytes	83/84	PLHD_LEN16	Both and data unreliable	Both
100 bytes	85~100	PLHD_LEN16	None	Both

It can be seen that the original intention of the early interrupt function is to do early detection at the header position of the data content. If the header data content does not meet the requirements, the receiving state can be exited in advance to save receiving time.

Near the critical value, the software application value of the early interrupt function is not great, and it is not recommended to use this function near the critical value.